NLP PROJECT REPORT

QUOTES RECOMMENDATION CHATBOT

Using Natural Language Processing & Rasa Framework

FRAMEWORK	LANGUAGE	TYPE
Rasa NLP	Python 3.8 - 3.10	Chatbot System
FRONTEND	**BACKEND**	**VERSION CONTROL**
HTML, CSS, JavaScript	Python Flask API	GitHub

SUBMITTED BY

Rajdeep Chaurasiya
Rishiraj Thakur
Satish Kumar Singh
Ronak Gupta

Student Project | Academic Documentation

1. Introduction

In today's fast-paced digital environment, individuals frequently experience stress, demotivation, and emotional imbalance due to academic, professional, and personal challenges. Although motivational quotes are widely available online, users must manually search through multiple websites or platforms to find content that matches their emotional state.

This project presents an AI-powered Quotes Recommendation Chatbot that leverages Natural Language Processing (NLP) and the Rasa framework to understand user intent or emotion and recommend relevant motivational quotes through natural conversation. The chatbot classifies user input into emotional categories such as motivation, love, success, and inspiration, then responds with appropriate quotes.

2. Problem Statement

The manual process of finding relevant motivational quotes online is time-consuming, non-personalized, and inefficient. Existing platforms such as BrainyQuote and Goodreads offer categorized quotes but rely entirely on manual browsing and keyword search, with no conversational interaction or real-time personalization.

As a result, users often struggle to quickly find quotes that truly match their feelings or current emotional state. Since these systems depend mainly on manual exploration, they cannot understand the user's intent or emotional context automatically. The absence of intelligent systems capable of analyzing user input and providing personalized recommendations reduces the effectiveness of existing quote platforms.

Key Challenges in Existing Systems:

- Time-consuming manual search process across multiple platforms
- Non-personalized results that don't match the user's current emotional state
- No conversational interface for natural interaction
- Inability to understand user intent or emotion automatically
- Lack of scalability for adding new content categories

3. Objectives

The primary objective of this project is to design and implement an intelligent chatbot system capable of understanding user emotions and recommending appropriate motivational quotes through natural conversation. The system aims to provide a more personalized and interactive way for users to access motivational content.

- Accept user input in natural language and classify the underlying intent or emotion
- Recommend quotes based on identified emotional category (motivation, love, success, hope, etc.)
- Provide interactive conversational responses using the Rasa NLU framework
- Implement a modular, scalable chatbot architecture for easy extension
- Deploy the chatbot via a web interface for real-world usability
- Train and evaluate the NLP model using structured training data

4. Social & Business Impact

The Quotes Recommendation Chatbot addresses a real emotional wellness need and has measurable social and business value. By delivering personalized motivational content through AI-driven conversation, it can positively impact mental well-being and user engagement.

This solution demonstrates how conversational AI can be applied to deliver meaningful content instantly, helping users receive encouragement and positive reinforcement through an interactive chatbot interface.

Impact Area	Description
Emotional Well-being	Provides personalized motivational support reducing stress and demotivation
User Engagement	Interactive conversation increases time-on-platform and user satisfaction
Scalability	Architecture supports addition of new quote categories and emotion types
Educational Use AI	Can be extended for wellness, mental health, and e-learning applications
Foundation	Serves as a foundation for advanced AI-powered recommendation systems

5. Literature Survey

Existing quote platforms such as BrainyQuote and Goodreads provide categorized quotes but rely on manual browsing and keyword search. These systems do not offer real-time personalization or conversational interaction. Conversational AI frameworks like Rasa enable intent classification and dialogue management using machine learning models such as the DIETClassifier. Most chatbot implementations focus on customer service or FAQ automation rather than emotion-based content recommendation. There is a clear gap in systems that combine conversational AI with personalized motivational content delivery a gap this project aims to address.

Recent advancements in Natural Language Processing and conversational AI have enabled the development of intelligent chatbot systems that can interact with users in natural language. By leveraging frameworks such as Rasa, it is possible to build scalable conversational agents capable of intent recognition, dialogue management, and real-time response generation.

Furthermore, the integration of web-based interfaces with conversational AI systems enables users to interact with chatbots through intuitive and accessible platforms. By combining Natural Language Processing with a responsive frontend and backend architecture, chatbot systems can deliver personalized content efficiently and in real time. This approach enhances user engagement and demonstrates the potential of AI-driven conversational systems in providing emotional support and motivational guidance.

6. Prerequisites & Environment Setup

6.1 Python Environment

Python3.8 - 3.10 is required. A virtual environment is strongly recommended.

```
python -m venv venv
source venv/bin/activate     # macOS/Linux
venv\Scripts\activate        # Windows
```

6.2 Rasa NLU Installation

```
pip install rasa
rasa --version
```

6.3 NLU Pipeline Configuration

The NLP pipeline is defined in config.yml and includes the following components:

Component	Purpose
WhitespaceTokenizer	Splits user input text into individual tokens
CountVectorsFeaturizer	Converts tokenized text into numerical feature vectors
DIETClassifier	Dual Intent and Entity Transformer performs intent classification
EntitySynonymMapper	Maps entity synonyms to a standard canonical value

6.4 Dataset Requirements

The chatbot requires a structured dataset containing quote text and corresponding emotion/category labels. Below is the expected CSV format:

quote	category
Never give up on your dreams	motivation
Love conquers all obstacles	love
This too shall pass	hope
Success is not final, failure is not fatal	success

7. Project Workflow

The development of the Quotes Recommendation Chatbot follows a structured milestone-based workflow covering problem definition, environment setup, model building, testing, and deployment.

Milestone 1	**Define Problem / Problem Understanding** Specify the business problem, define requirements, conduct literature survey, and analyse social and business impact.
Milestone 2	**Environment Setup** Install Rasa and all required dependencies. Set up the standard Rasa project directory structure.
Milestone 3	**Data Collection & Model Building** Create user intent training data (nlu.yml), define bot responses (domain.yml), create conversation stories (stories.yml), and train the chatbot model.
Milestone 4	**Testing & Deployment** Test model using Rasa shell and test stories. Deploy chatbot using a web interface and validate the deployed chatbot.

The project workflow follows a structured development approach where each milestone focuses on a specific phase of building the chatbot system. Initially, the problem and requirements are clearly defined to understand the need for an automated quote recommendation system. After identifying the problem, the development environment is prepared by installing the required tools and setting up the Rasa framework.

In the next stage, training data is created to enable the chatbot to understand different user intents. The Natural Language Processing model is trained using user examples, conversation stories, and predefined responses. Once the model is trained, the chatbot is tested to ensure it correctly identifies user intent and provides appropriate quote responses.

Finally, the chatbot is integrated with a web-based interface and deployed so that users can interact with it through a conversational chat system. This structured workflow ensures that the chatbot is developed, tested, and deployed in an organized and efficient manner.

In addition, continuous testing and validation are performed throughout the workflow to ensure that the chatbot responds accurately to different types of user inputs. Test stories and evaluation metrics help verify that the system correctly identifies intents and generates appropriate responses. This iterative testing process improves the chatbot's reliability and ensures a smooth user experience when interacting with the system.

8. Project Structure

The project follows the standard Rasa directory structure:

```
quotes-recommendation-chatbot/
    ■■■ data/
        ■■■ nlu.yml
        ■■■ stories.yml
    ■■■ domain.yml
    ■■■ config.yml
    ■■■ credentials.yml
    ■■■ endpoints.yml
    ■■■ models/
    ■■■ README.md
```

File / Folder	Description
`data/nlu.yml`	User intents and NLP training examples
`data/stories.yml`	Conversation flow definitions and dialogue paths
`domain.yml`	Intents, entities, bot responses, and actions
`config.yml`	NLU pipeline configuration (tokenizer, classifier, etc.)
`models/`	Directory storing all trained chatbot model files
`credentials.yml`	External channel credentials (Slack, Telegram, etc.)
`endpoints.yml`	Action server and other endpoint configurations

The project structure follows the standard Rasa framework organization to maintain modularity and ease of development. Each folder and configuration file plays a specific role in managing the chatbot's training data, dialogue flow, and system configuration. The data directory stores the training examples for user intents and conversation stories, which help the chatbot understand user inputs and determine appropriate responses.

The domain.yml file defines the core elements of the chatbot including intents, responses, and actions. The config.yml file contains the Natural Language Processing pipeline configuration used for intent classification and message processing. Meanwhile, the models directory stores the trained machine learning models generated after training the chatbot.

The configuration files such as credentials.yml and endpoints.yml are used to manage external integrations and action server communication. This structured organization ensures that the chatbot project remains scalable, maintainable, and easy to extend with additional intents, responses, or conversation flows in the future.

9. Rasa Training Data Format

9.1 NLU Intent Examples (nlu.yml)

Training data must follow the Rasa YAML structure, mapping intents to example user utterances:

```
version: "3.1"

nlu:
- intent: sadness
examples: |
    - I feel lonely
    - I am sad today
    - I feel low and hopeless

- intent: motivation
examples: |
    - Inspire me
    - I need motivation
    - Encourage me please

- intent: love
examples: |
    - Share a love quote
    - I want something about love
```

9.2 Supported Intent Categories

Intent	Example Triggers	Quote Theme
motivation	I need motivation / inspire me	Achievement, perseverance
sadness	I feel sad / I am low	Hope, healing, recovery
love	Share a love quote	Love, relationships
success	Success quotes / how to succeed	Success, ambition
inspiration	Inspire me / I need inspiration	Life, wisdom
funny	Tell me something funny	Humor, laughter

10. System Features

✓ **Emotion-based Recommendation**	Understands user emotions and recommends quotes that match the user's current mood or emotional state.
✓ **Intent Recognition (NLU)**	Uses Rasa NLU with DIETClassifier to accurately classify user intent from free-form natural language input.
✓ **Structured Conversation Flow**	Manages multi-turn dialogue using stories and rules, ensuring smooth and coherent conversation paths.
✓ **Custom NLU Pipeline**	Configurable NLP pipeline with WhitespaceTokenizer, CountVectorsFeaturizer, and EntitySynonymMapper.
✓ **Scalable Architecture**	Modular design allows easy addition of new quote categories, intents, and response templates.
✓ **Web Interface Deployment**	Deployable via Rasa shell or integrated into a web frontend for browser-based interaction.
✓ **Category Quick-Select**	Users can click category chips (Motivation, Love, Success, Funny) for fast quote retrieval.

11. How to Run the Project

Step 1: Clone the Repository

```
git clone https://github.com/your-username/quotes-recommendation-chatbot.git
cd quotes-recommendation-chatbot
```

Step 2: Activate Virtual Environment

```
source venv/bin/activate # macOS/Linux
venv\Scripts\activate # Windows
```

Step 3: Train the Model

```
rasa train
```

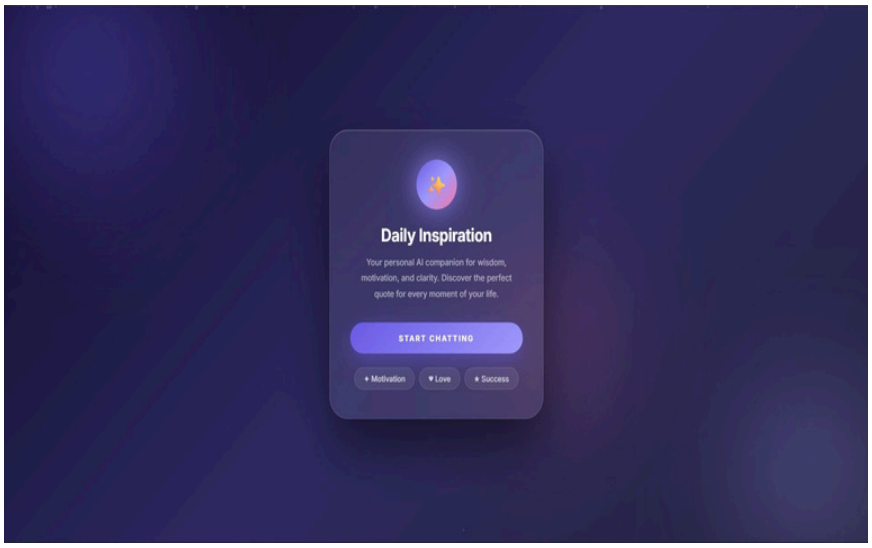
Step 4: Run the Chatbot

```
rasa shell
```

12. Application Screenshots

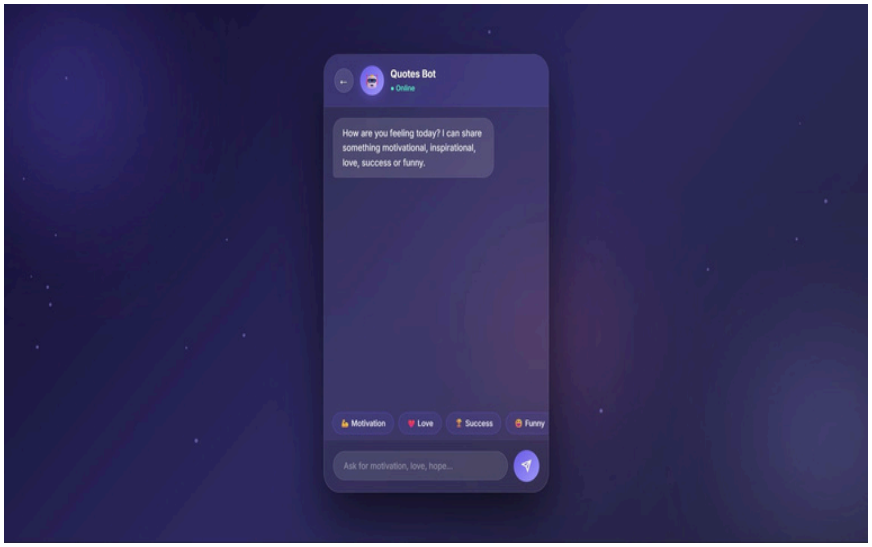
12.1 Welcome / Landing Screen

Thelanding screenpresents the'Daily Inspiration' interface with a clean dark-themed design. Users can click 'Start Chatting' or use quick-access category chips (Motivation, Love, Success) to begin their quote discovery journey.



12.2 Chat Interface

The chat interface features the 'Quotes Bot' header with an online status indicator. The bot greets users with an emotion check-in and offers category buttons (Motivation, Love, Success, Funny). Users can also type freely in the input field at the bottom.



13. Technologies Used

Technology	Purpose	Version
Python	Core programming language for all backend logic	3.8 - 3.10
Rasa	NLU framework for intent classification and dialogue management	Latest
DIETClassifier	Dual Intent and Entity Transformer for NLP classification	Built-in
WhitespaceTokenizer	Text tokenization component in the NLU pipeline	Built-in
CountVectorsFeaturizer	Text-to-vector feature extraction for model training	Built-in
YAML	Data format for training data, domain, config, and stories files	3.1
Git / GitHub	Version control and source code repository management	Latest
VS Code	Recommended IDE for development and debugging	Latest

14. Conclusion

The Quotes Recommendation Chatbot successfully demonstrates how Natural Language Processing and the Rasa framework can be used to build an intelligent, emotion-aware conversational system. The project follows a structured development lifecycle covering problem understanding, environment setup, model training, testing, and deployment.

By using DIETClassifier for intent classification and a structured NLU pipeline, the chatbot can accurately identify user emotions and respond with appropriate motivational quotes. The modular architecture ensures the system is easily extensible for future enhancements.

15. Future Work

- Integrating a React or Flask web frontend for a rich browser-based chat experience
- Connecting to a live database of curated quotes with tagging and author information
- Adding sentiment analysis to detect emotion from tone, not just intent keywords
- Deploying the system on cloud platforms (AWS, Google Cloud, or Heroku) for scalability
- Implementing user session memory for personalized quote history and preferences
- Adding support for regional languages to make the chatbot more inclusive
- Incorporating a feedback mechanism for users to rate recommended quotes
- Extending to mental health support applications with professional escalation pathways

This document was prepared as part of an academic project on NLP-based conversational AI systems.
All implementation details are based on the Rasa open-source framework.